

Промежуточный отчёт по дополнительному заданию по предмету “Технологии управления данными”

Текст задачи:

По согласованию с преподавателем можно взять дополнительное задание – реализация программного модуля поддержки миграции данных. Дополнительное задание может быть оценено в 20 баллов.

Решение:

Манипуляция контейнерами docker и выполнение скриптов, созданных в лабораторных 1, 4, 5, 6 выполняется через встроенную в Python библиотеку subprocess

Работа с базами данных выполняется через библиотеку psycopg2

Парсинг аргументов для удобных команд выполняется через встроенную в Python библиотеку argparse

Код главного скрипта main.py (на Python):

```
import subprocess
import argparse

class MyException(Exception):
    pass

class MyArgumentParser(argparse.ArgumentParser):
    def error(self, message):
        raise MyException(message)

def run_docker(db_name, m):
    filename = db_ymls[db_name]
    command_param = ["docker", "compose", "-p", f"postgres-{db_name}", "-f",
filename, m]
    new_process = subprocess.Popen(command_param)
    return new_process

def run_python(filename):
    command_param = [python_exe, filename]
    new_process = subprocess.Popen(command_param)
    new_process.wait()

def run_sql(db_name, mode):
    command_param = [python_exe, "sql_runner.py", db_name, mode]
    new_process = subprocess.Popen(command_param)
    new_process.wait()

python_exe =
"C:\\Users\\ajtal\\AppData\\Local\\Programs\\Python\\Python311\\python.exe"

db_ymls = {
    "west": "C:\\Users\\ajtal\\Desktop\\all\\база (ха-ха) данных\\3
year\\lab1\\docker-compose-west.yml",
```

```

    "east": "C:\\Users\\ajtal\\Desktop\\all\\база (ха-ха) данных\\3
year\\lab1\\docker-compose-east.yml",
    "ware": "C:\\Users\\ajtal\\Desktop\\all\\база (ха-ха) данных\\3
year\\lab2\\docker-compose.yml",
    "mart": "C:\\Users\\ajtal\\Desktop\\all\\база (ха-ха) данных\\3
year\\lab3\\docker-compose.yml",
}

scripts = {
    "seeding": "C:\\Users\\ajtal\\Desktop\\all\\база (ха-ха) данных\\3
year\\lab1\\seeding.py",
    "export": "C:\\Users\\ajtal\\Desktop\\all\\база (ха-ха) данных\\3
year\\lab4\\main.py",
    "report": "C:\\Users\\ajtal\\Desktop\\all\\база (ха-ха) данных\\3
year\\lab5\\main.py",
    "restore": "C:\\Users\\ajtal\\Desktop\\all\\база (ха-ха) данных\\3
year\\lab6\\main.py",
}

help_message = """
[database | db | script | s | exit | end | help] - Subcommands

database (db) :
    manipulates databases
    [database | db] [mode] {dbs}
    mode - mode of operation {up, create, stop, delete, down}:
        up - raises docker containers
        create - creates tables
        stop - stops docker containers
        delete - drops tables
        down - lowers docker containers
    dbs - names of databases to perform operations on {west, east, ware,
mart}

perform (p) :
    performs operations on databases
    [perform | p] {operation}
    operation - operation to perform on databases {seeding, export, report,
restore}:
        seeding - creates fake information for east and west databases
        export - exports data from east and west into ware database
        report - creates report from ware for mart database
        restore - restores data for east and west databases from ware

exit (end) :
    exits the application
    [exit | end]

```

```

help          :
    prints this help message
    help
    """

if __name__ == "__main__":
    parser = MyArgumentParser(add_help=False)
    subparsers = parser.add_subparsers(required=True)

    database_parser = subparsers.add_parser("database", aliases=["db"],
add_help=False)
    database_parser.add_argument("mode", nargs="?", choices=["up", "stop",
"down", "delete", "create"])
    database_parser.add_argument("dbs", nargs="+", type=str)
    database_parser.set_defaults(subcommand="database")

    script_parser = subparsers.add_parser("perform", aliases=["p"],
add_help=False)
    script_parser.add_argument("script", type=str)
    script_parser.set_defaults(subcommand="perform")

    exit_parser = subparsers.add_parser("exit", aliases=["end"],
add_help=False)
    exit_parser.set_defaults(subcommand="exit")

    help_parser = subparsers.add_parser("help", add_help=False)
    help_parser.set_defaults(subcommand="help")

    processes = []
    confirming = ""
    while True:
        try:
            args = parser.parse_args(input().split())
            if args.subcommand == "database":
                if args.mode == "delete":
                    confirming = input("Type 'confirm' to delete data: ")
                for db in args.dbs:
                    if args.mode != "delete" and args.mode != "create":
                        processes.append(run_docker(db, args.mode))
                    if args.mode == "create":
                        run_sql(db, "create")
                    elif args.mode == "delete":
                        if confirming == "confirm":
                            run_sql(db, "drop")
                if args.mode == "create" or args.mode == "delete":
                    print("Complete")
            elif args.subcommand == "perform":
                run_python(scripts[args.script])

```

```

        elif args.subcommand == "exit":
            break
        elif args.subcommand == "help":
            print(help_message)
    except (IndexError, KeyError, ValueError, MyException):
        print("Invalid command, type 'help' for information")
for p in processes:
    p.terminate()

```

Код вспомогательного скрипта sql_runner.py (на Python):

```

import argparse
import psycopg2

db_config_west = {
    "dbname": "west_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5446"
}

db_config_east = {
    "dbname": "east_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5447"
}

db_config_warehouse = {
    "dbname": "warehouse_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5448"
}

db_config_mart = {
    "dbname": "mart_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5449"
}

configs = {
    "west": db_config_west,
    "east": db_config_east,

```

```

    "ware": db_config_warehouse,
    "mart": db_config_mart,
}

sqls = {
    "west": "lab1.sql",
    "east": "lab1.sql",
    "ware": "lab2.sql",
    "mart": "lab3.sql"
}

def run_sql(db_name, mode):
    config = configs[db_name]
    sql_filename = f"C:\\Users\\ajtal\\Desktop\\all\\база (ха-ха) данных\\3
year\\control\\sql\\{mode}\\{sqls[db_name]}"
    with open(sql_filename, "r") as sql_file:
        sql = sql_file.read()
        conn = psycopg2.connect(**config)
        with conn.cursor() as cur:
            cur.execute(sql)
            conn.commit()

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("db")
    parser.add_argument("mode", choices=["create", "drop"])
    args = parser.parse_args()
    run_sql(args.db, args.mode)
    print("SQL Done")

```

Также были скопированы*/созданы sql-скрипты для создания/удаления таблиц в базах данных.

*из лабораторных 1-3 соответственно